

Constructor: Final Design Document

Team: Muneeb, Alex, Laith
CS246 Spring 2026 – Due Date 2
July 28, 2026

Introduction

Constructor is a four-player strategy game set at the University of Waterloo, adapted from Settlers of Catan. Builders compete to develop student housing across a board of nineteen resource tiles, collecting BRICK, ENERGY, GLASS, HEAT, and WIFI as dice rolls activate the tiles their residences touch. Those resources buy roads and basements, and a basement may later be improved into a house and then a tower, each upgrade increasing both the builder's score and the yield that tile pays out. The first builder to reach ten building points wins. A flock of geese, relocated by whichever builder rolls a seven, suppresses production on the tile it occupies and allows that builder to steal a resource from a neighbour.

This document describes the design of our implementation. Three concerns shaped it: a strict separation between the game model, its presentation, and the code that drives a turn; the use of design patterns only where a decision genuinely varies at runtime; and ownership expressed entirely through RAII, so that the program performs no explicit memory management anywhere.

Overview

The program is roughly 1800 lines of C++20 across 26 source files, organised with modules rather than headers. The module structure follows the layering of the design.

- Five modules hold shared vocabulary types with no behaviour: `material`, `colour`, `residenceLevel`, `tileInfo`, and `subj_obs`, the last containing the `Observer` and `Subject` base classes. These sit at the bottom of the dependency graph and depend on nothing.
- Two modules hold interchangeable strategies: `boardSource`, which decides how a board is laid out, and `dice`, which decides how a roll is produced. Each is an abstract base with concrete subclasses, and neither knows that the board or the builders exist.
- One module, `board`, holds the game model, divided into six partitions: `:settlement`, `:vertice`, `:edge`, `:tile`, `:map`, and `:player`. These six types refer to one another directly, so they belong in a single module; partitions keep each in its own file.
- One module, `interface`, holds the controller: command parsing, the turn loop, command-line handling, and save and load.

The architecture is Model-View-Controller. `Map` and the objects it owns, together with `Player`, form the model. The view is distributed rather than centralised: every model class implements a `display()` method returning a `string`, or for `Tile` a small `TileInfo` record, and `Map::display()` assembles these into the board drawing. No model class writes to an output stream. `Interface` is the controller and the only class in the program that reads input or produces output during play. This separation is

what allows the whole model layer to be driven by a different front end without modification, and it is discussed further under Resilience to Change.

Design

Ownership and memory management

Every owning relationship is expressed through a vector or a `unique_ptr`, and the program contains no `new` and no `delete`. `Interface` owns the `Map` through a `unique_ptr<Map>` and the four builders through a `vector<unique_ptr<Player>>`. `Map` owns its nineteen tiles, seventy-two edges, and fifty-four vertices in three vectors held by value. A `Vertex` exclusively owns whatever `Settlement` is built on it, through a `unique_ptr<Settlement>`. Every remaining pointer in the program is a raw pointer chosen deliberately to express non-ownership: `Edge::road` and `Settlement::player` identify the builder who owns a piece, `Map::goosed` marks the tile currently holding the geese, and the cross-references between `Edge` and `Vertex` describe board adjacency. Destruction is entirely automatic and correctly ordered by construction.

One consequence of this arrangement drove a specific decision. Because settlements and edges hold raw pointers to builders, the container holding those builders must never reallocate once such a pointer has been taken. Storing them as `vector<Player>` by value would invalidate every one of those pointers the first time the vector grew. Holding them as `vector<unique_ptr<Player>>` means the vector relocates pointers rather than `Player` objects, so the references remain valid for the life of the game. The same reasoning governs `Map`: its three vectors are filled to final size during construction, before any tile, edge, or vertex takes a pointer into them, and are never resized afterwards.

Representing the board

The board is a graph rather than a grid. `Map` holds three flat vectors and wires them together at construction: each edge stores the two vertices it joins, each vertex stores the two or three edges meeting at it, and each tile stores the six edges and six vertices that surround it. The specification observes that the board's shape is symmetric but not in a way that yields a simple formula, and suggests either hard-coding the relationships or reading them from a file. We generate them instead from a small set of functions describing which columns are occupied on each row, which reproduces the numbering of Figure 3 exactly without a hand-written table of adjacency lists.

Because every question about legality reduces to a walk over this graph, the rules that depend on geometry live on `Map`, the only class able to see the whole structure: whether a road may be built on an edge, whether a residence may be built at a vertex, which builders hold residences on a given tile, and the longest-road search described later.

Distributing resources: the Observer pattern

When a builder rolls anything other than a seven, every tile whose value matches must pay each residence standing on it. Rather than have the map inspect tiles and vertices to work out who is owed what, `Map` derives from `Subject` and each `Tile` from `Observer`, with every tile registering itself at construction. A roll becomes a single call to `Map::notify(rollValue)`, which invokes `Tile::update` on all nineteen tiles. A tile whose value does not match, or which is the Park, or which

currently holds the geese, does nothing. A tile that does match walks its own six vertices and asks each to pay, and the settlement there credits its owner an amount determined by its level. The map never learns how a tile decides to pay out, and a tile never learns what caused the roll.

Interchangeable decisions: the Strategy pattern

Two decisions in the game genuinely vary at runtime, and both are implemented as Strategy.

BoardSource is an abstract class with one method, `getTiles()`, returning a `vector<TileInfo>`. FileBoardSource parses a layout file; RandomBoardSource generates a layout with the resource and value distribution the specification prescribes. Interface selects the appropriate one from the command line and hands it to Map's constructor, which calls `getTiles()` without knowing which kind it holds.

Dice is an abstract class with one method, `roll()`. LoadedDice prompts the builder and validates the response; FairDice sums two pseudo-random values. Each Player holds a `unique_ptr<Dice>`, replaced by `Player::setDice` in response to the `load` and `fair` commands. `Player::rollDice()` is a single delegating call, and no command handler ever branches on which kind of dice a builder is using.

Residences: a level field rather than a hierarchy

Basements, houses, and towers differ in cost, in building points, in how much they collect, and in how they are drawn, which makes an inheritance hierarchy an obvious first thought. We chose a single `Settlement` class carrying a `ResidenceLevel` field instead, because an object's dynamic type in C++ is fixed at construction. Were the three levels subclasses, improving a residence would require destroying one object and constructing a different one in its place, invalidating the `unique_ptr<Settlement>` held by the vertex and breaking the rule that a residence keeps its identity and its owner across an upgrade. With a level field, an improvement advances an enumeration and every level-dependent quantity is derived from it, so cost, score, and display can never disagree with one another.

Where the rules live

Legality and mutation are deliberately separated. Map answers the questions that require seeing the whole board: whether a vertex is adjacent to an existing residence, and whether a builder has a road reaching a given edge. Edge and Vertex enforce the local invariants only they can see, such as whether an edge is already built or a residence is already a tower, and perform the mutation itself. Player sits between them and always checks before acting: a request to build a road asks the map whether it is legal, and only then asks the edge to build. Nothing in the type system enforces that ordering, so it is a convention we agreed on explicitly and hold to throughout.

Two contexts complicate this arrangement, and both are handled by an explicit parameter rather than by a second code path. The opening basements of the setup phase are chosen rather than purchased, and are exempt from the rule that a residence must stand beside one of the builder's own roads; the building routines therefore take a flag distinguishing setup from ordinary play. Restoring a saved game raises the same problem in a different form, since roads must be rebuilt at a point where the builders' resources have not yet been credited, so road construction takes a corresponding flag. Both flags

default to ordinary behaviour, which keeps the common call sites unchanged and confines the special case to the two callers that need it.

Error handling

Every operation that can legitimately fail returns `bool` and leaves state untouched when it does. The controller checks the result and prints the message the specification requires, and play continues. The reasoning behind this choice is set out under Question 7.

Coupling and cohesion

The dependency graph is close to acyclic. The vocabulary modules depend on nothing. The two strategy modules depend only on vocabulary types, and in particular neither knows that `Map` or `Player` exists, which is exactly what makes them substitutable. The model module depends on the vocabulary and strategy modules; the controller depends on everything below it, and nothing depends on the controller. The single deliberate exception is that `Player` holds a reference to `Map` so that it can act on the board, which is why the six model types form one module with partitions rather than six separate modules, since C++20 forbids the circular imports that separating them would require.

Each class has one reason to change. `Tile` knows a tile's resource, value, and geese state, and nothing about builders. `Settlement` knows one residence's level and owner, and nothing about the board. `Interface` knows about input, output, and turn order, and nothing about how a road decides whether it may be built. The clearest evidence of the separation is that the model layer contains no output statements at all: `display()` methods return strings, and the controller decides what becomes of them.

One piece of coupling we accepted knowingly is that the controller recovers the vertex number of a residence by scanning the map's vertices, because a `Settlement` holds no back-pointer to the vertex beneath it. Adding that pointer would remove the scan at the cost of another cross-reference to keep consistent, and across fifty-four vertices the scan costs nothing worth optimising.

Relationship to the original class model

The class model we designed before implementation proved structurally sound: no class was added, removed, or repurposed during development, and no relationship between classes changed. Where signatures needed widening, it was because they had been drawn more narrowly than a specification requirement could be met with, and the pattern is instructive. Losing half a hand to the geese and stealing from a neighbour were first modelled as operations that mutate a builder and return either a count or nothing at all, but the specification requires naming each resource type lost and the resource stolen, which neither could supply. Both now return what they actually transferred. An operation that mutates state must also be able to describe what it did, if anything downstream is expected to report on it.

The remaining adjustments added the accessors the controller needs: a builder's point total, required by both the status line and the win condition, and the index of the tile holding the geese, required by the save format. The point total is computed from the builder's residences on each call rather than stored, so that it cannot fall out of step when a residence is improved.

Resilience to Change

The specification could plausibly change in several directions, and the design was shaped around making each of them cheap. In each case we state what would actually have to be edited.

A new way of setting up the board. Adding a source, whether a network layout, a procedurally generated variant, or a fixed tutorial board, means one new class deriving from `BoardSource` and one new branch in the controller's argument parsing. `Map` does not change at all, because it knows only that it has been handed something it can call `getTiles()` on. This is precisely why the pattern is there rather than a conditional inside the map's constructor.

A new kind of dice. The same shape: one new class deriving from `Dice`, one new command in the controller. `Player` is untouched, since it holds only a `unique_ptr<Dice>` and calls `roll()`.

A new command. Commands are dispatched in one place, and each resolves to operations the model already exposes. The two enhancement commands described below were added exactly this way and required no change to any model class.

A new resource type. Resources are a single enumeration, and a builder's holdings are reachable as an ordered vector through one accessor. A sixth resource means an enumeration value, an extension to that vector and the name parser, and revised cost tables. Nothing outside `Player` and those tables inspects which resource it handles: tiles, vertices, and settlements pass a `Material` through without examining it.

A new residence level. Because a residence is one class with a level field rather than a hierarchy, a fourth level means an enumeration value and three small extensions to the level-to-value mappings for cost, points, and display character. There is no new class, no change to `Vertex`, and no change to how ownership or upgrades are tracked.

A different display, or a graphical front end. This is the change the layering protects against most deliberately. No model class writes output, and `Tile::display()` returns structured data rather than formatted text precisely so that a caller may render it differently. A graphical or full-screen front end would be a second controller consuming the same `display()` results, with the entire model reused unchanged. This is the direction discussed under Question 3.

A change to the building rules. The adjacency rules live in two methods on `Map`. Altering the spacing required between residences, or permitting roads to pass through an opponent's residence, means editing one method each. Nothing else in the program holds an opinion on the matter, because nothing else performs adjacency reasoning.

Turning features on and off. Both enhancements are gated behind a single boolean held by the controller and switchable during play rather than only at startup. The mechanism generalises: a feature with a command of its own is disabled by declining to recognise the command, and a feature without one is disabled by passing the flag into the calculation it affects.

The table below summarises where the design is strong and where it is weaker.

Change	Cost	What changes	What does not
New board source	Very low	One new subclass; one argument branch	Map, model layer, all rules

Change	Cost	What changes	What does not
New dice type	Very low	One new subclass; one command	Player, model layer
New command	Very low	One dispatch branch	Entire model layer
New residence level	Low	Enum value; three level mappings	Vertice, ownership, upgrades
New resource type	Low	Enum value; resource vector; cost tables	Tile, Vertice, Settlement
Building rule change	Low	One or two methods on Map	Everything else
Graphical front end	Medium	A second controller	Entire model layer
Different player count	Medium	Several loops in the controller	Model layer

The direction we are least well placed for is a different number of builders: four is assumed in several places in the controller. Nothing in the model layer would change, but the work is spread across more call sites than anything else on this list.

Answers to Questions

The specification poses seven questions. The first two concern decisions the program makes at runtime, and are answered by patterns present in the delivered code. The remaining five concern extensions we designed for but did not build, and each explains both the approach and why we judged it outside the scope of this specification.

Question 1: choosing between a random board and a file at runtime

We use the Strategy pattern. `BoardSource` is abstract with a single `getTiles()` method, implemented by `FileBoardSource` and `RandomBoardSource`. The controller selects one from the command line and passes it to `Map`'s constructor, which never learns which it received. The strategy object is owned only during setup and released once the board exists, since nothing needs it afterwards. This is the pattern we anticipated using, and the implementation follows it directly.

Question 2: switching between loaded and fair dice at runtime

Strategy again. `Dice` is abstract with a single `roll()`, and each builder holds a `unique_ptr<Dice>` that `setDice` replaces in response to the `load` and `fair` commands.

The pattern depends entirely on one implementation detail. `Dice::roll` was initially declared without `virtual`, which meant that calls through the `unique_ptr<Dice>` resolved to the base class and never reached either subclass. The pattern was correct on paper and inert in practice. Declaring `roll()` pure virtual and giving `Dice` a virtual destructor fixed it. Strategy depends on dynamic dispatch actually being enabled, and a polymorphic base class without a virtual destructor is a defect whether or not it has yet produced a visible leak.

Question 3: supporting different game modes

Our answer is Strategy applied at the view and controller layer. A hexagonal board is a different `BoardSource` combined with a different adjacency-generation step in `Map`'s constructor, both of which are already isolated. A graphical display is a second controller: because no model class writes output,

and because `Tile::display()` already returns structured data rather than a formatted string, a new front end would consume the same model unchanged. Making that concrete would mean promoting `Interface` to an abstract base with command-line and graphical implementations beneath it. We chose not to, because the specification requires exactly one interface, and an abstraction with a single implementation adds indirection while removing no duplication. Different player counts are the one item here we are less well positioned for, as discussed above.

Question 4: guaranteeing a legal action sequence across player types

Our answer is the Template Method pattern with the Non-Virtual Interface idiom. The controller already encodes the legal sequence of a turn in two non-public methods that run in a fixed order. Introducing computer players would mean promoting that sequence into a non-virtual `takeTurn()` on an abstract `PlayerController`, with the individual decisions inside it, such as which vertex to build on or whether to accept a trade, exposed as private virtual methods overridden by human and computer subclasses. Because `takeTurn()` would be non-virtual, no subclass could reorder or omit a step, which is exactly the guarantee the question asks for. We did not build it because the specification has only human players, leaving the abstraction with a single implementation.

Question 5: computer players with different strategies

This composes with the previous answer rather than replacing it. A computer controller would hold a `unique_ptr<AIStrategy>` whose methods correspond to the decision points the template method already delegates, with easy, medium, and aggressive implementations behind them. Changing difficulty during play would be a single pointer reassignment, exactly the mechanism `setDice` already uses for dice. Nesting Strategy beneath Template Method keeps the sequence of a turn fixed while leaving the decisions within it free to vary, and avoids the combinatorial growth that would follow from encoding each difficulty as its own controller subclass.

Question 6: changing tile production during a game

Our answer is the Decorator pattern. A tile's production step would be wrapped by decorators implementing the same interface, one adding a second resource type to whatever the wrapped tile yields and another reducing the quantity on each successive trigger, each delegating to the object it wraps. This avoids a dedicated subclass for every combination of modifiers a tile might accumulate over a game. `Tile::update` is the natural wrapping point, since it is already the single place where a tile decides what to pay. We did not build it because tile production is fixed for the whole game in this specification, with no mechanism by which a builder could alter it.

Question 7: exceptions

Almost every failure in Constructor is ordinary. A builder asks to build on an occupied vertex, proposes a trade they cannot afford, mistypes a command, or enters a roll outside the legal range. The specification requires a particular message and continued play for each of these, which is precisely the situation the course notes describe as suited to error codes rather than exceptions: failure is anticipated, the caller is expected to check for it, and no invariant has been compromised. Every such operation therefore returns `bool`, and the controller turns the result into the required output.

Exceptions appear in one place only, at a boundary where the standard library leaves no alternative. The two helpers that convert a command argument into an integer call `std::stoi`, which throws on input it cannot parse; each wraps that call and converts the exception immediately into a `false` return, so nothing propagates beyond the function that produced it. The `istreamstream` extraction idiom from the course notes would remove even this, reporting failure through stream state instead, and is what we would use given the chance to revisit that code. Our plan of attack anticipated avoiding exceptions entirely; this parsing boundary is the one place the delivered program does not.

File input at startup is the one case where an exception might genuinely have been warranted. A missing or malformed save or layout file is arguably exceptional rather than routine, and the specification defines no message for it. We check the stream's failure state once and treat a failed load as a `false` return, on the view that introducing a custom exception type for a case the specification does not require us to handle gracefully would add machinery without adding safety.

Extra Credit Features

Memory management without explicit deallocation

The program contains no `new`, no `delete`, and no C-style allocation anywhere. All ownership is expressed with `vector` and `unique_ptr` as described earlier, and every remaining raw pointer is deliberately non-owning. The challenge in this project was less about avoiding `delete` than about deciding where ownership belonged in a structure that is densely cross-referenced: a vertex owns its settlement, but a settlement points back at a builder it does not own; a map owns its edges, but an edge points at vertices it does not own, and at a builder owned elsewhere entirely. Getting those directions right is what made the smart pointers sufficient rather than merely decorative.

We verified the result dynamically rather than by inspection alone. Our regression suite of eighty scripted sessions, covering the opening placements, full turn cycles, resource distribution, trading, the geese and stealing, saving and reloading, both enhancements, and a game played through to a win and restarted, was run in its entirety under `valgrind --leak-check=full`. Every case reports that all heap blocks were freed and that no errors were detected.

Bank trading

A builder may trade with the bank at a fixed four-to-one ratio through a `bank` command available during the same phase as ordinary trading. It reuses the existing material parser and the existing resource adjustments on `Player`, and needed no new method on any model class. Insufficient resources produce the same message the specification already defines for that situation. We chose a flat ratio rather than ports because the board model in this specification has no notion of a port, and inventing one would have required adjacency data with no basis in the rules.

Longest road

The builder holding the longest continuous chain of their own roads receives two additional building points, provided the chain is at least five roads long. The bonus transfers only when another builder strictly exceeds the current holder, so a tie leaves it where it is, which matches the original game and prevents it oscillating between builders.

The interesting part of this feature is the definition of the chain itself. Longest road for a builder is the longest simple path through that builder's own edges, computed by a depth-first search with backtracking run from every vertex, which is comfortably affordable across seventy-two edges. One rule complicates it and is easy to overlook: a path may pass through a vertex that is empty or holds that builder's own residence, but is blocked from continuing through a vertex holding an opponent's residence. Omitting that check produces a plausible but incorrect answer in ordinary board positions, so it is implemented rather than simplified away. The holder is recomputed after a road is built and after a save is restored, not on every query, since building a road is the only event that can change it.

The bonus is tracked on Map rather than on PLayer, because the map is the only class that can see every builder's roads, whereas a builder has no access to another builder's state. A builder's point total asks the map whether it currently holds the bonus.

Enabling and disabling enhancements

Both features sit behind a single switch. The guidelines require an enhancement to be selectable by a command-line flag on the same binary rather than by compiling two versions, and note that it would be better still if enhancements could be turned on and off while the program runs. We implemented the second form. A flag sets the initial state, and two commands change it at any point during play. Those commands are always recognised, whatever the current state, since gating them behind the switch they control would make the feature impossible to enable. With the switch off, the bank command is rejected exactly as any unrecognised command would be, and building-point totals are computed exactly as they are in the unmodified game.

Final Questions

What this project taught us about developing software in teams

The most valuable thing we did was settle the interfaces before writing the code behind them. Because the class model fixed the exact signature and ownership semantics of every class in advance, three people could write against each other's classes before those classes existed. Where we later departed from that model, the cost was immediate and visible.

Getting from an agreed design to working code without stepping on one another took deliberate process rather than good intentions. We met on a regular schedule and talked each piece through before anyone began building it, which is what kept three people implementing against one shared model rather than three subtly different readings of it. Work happened on separate feature branches, and nothing reached the main branch until the group had discussed it and agreed unanimously. Git carried most of this: it let us see precisely what had changed and when, and made working in parallel possible without one person's half-finished work blocking anyone else's.

A shared design is only useful while it remains true. We kept a running log of every divergence between the model and the code, together with the rule that forced it. This began as bookkeeping and became the most useful process decision we made: a signature changed by one person to satisfy a requirement reached the others as a written justification rather than as a surprise during integration. Several entries, including the flags separating setup from ordinary building, were changes none of us anticipated at design time and would have produced conflicting assumptions had they gone unrecorded.

Integration failures cluster at the seams rather than inside the components. Our most instructive defect was not a wrong algorithm: loading a saved game silently restored no roads, because the routine rebuilding them charged each road its cost at a point where the builders' resources had not yet been restored, and the failed return value went unchecked. Each half of that was locally correct; only the combination was wrong, and it was invisible without a test that saved a game and reloaded it. The boundary between two people's correct work is where the defects live, and ignoring a `bool` return is how they stay hidden.

Finally, keeping the build working from the first day was worth more than any single technique. We stubbed every class from the model with empty bodies before implementing anything, so the project compiled and ran immediately and never stopped doing so. That meant we always had something submittable, and it meant a compilation failure always pointed at the change we had just made rather than at an accumulated pile. We kept the surrounding setup deliberately light for the same reason, working in the shared CS246 student environment with the course-provided compiler rather than each of us maintaining a private Linux toolchain. That removed an entire category of problem before it could arise: when something failed to build, it failed identically for all three of us, and no one lost an evening to a difference that existed only between our machines.

What we would do differently

We would write the scripted tests far earlier. For most of the project we tested by playing the game by hand, which is slow, inconsistent between runs, and effectively impossible to repeat after a change. Late in the project we moved to feeding recorded command sequences through standard input with a fixed random seed, which made a full session reproducible and made memory checking straightforward. Every significant defect we found late would have been caught earlier and more cheaply by a handful of those scripts, and the save-and-reload defect above would have surfaced the first time such a script existed.

We would also treat the required output text as data rather than as literals scattered through the controller. Because the specification is precise about wording, a single misplaced word in a prompt is a real defect, and we carried one for a while. Collecting the required strings in one place would have made them checkable against the specification in a single pass rather than by reading the whole controller.

On the design itself, we would give a settlement a back-pointer to the vertex it occupies. Leaving it out kept the object smaller, but the controller now recovers vertex numbers by scanning the board in two separate places, which is more code in a more awkward position than the field would have been. We would also decide the ownership model for the builder collection before writing anything that points into it; changing it afterwards was mechanical but touched more of the program than it should have.

Conclusion

Constructor is complete against the specification, with three enhancements beyond it. The design decisions we are most confident in are the ones that isolated change: the two Strategy hierarchies that keep board setup and dice generation substitutable, the Observer relationship that leaves the map and its tiles ignorant of one another's internals, the choice to model a residence as one class with a level rather than as a hierarchy, and the separation between the classes that decide and the controller that

reports. The decisions we would revisit, and the reasoning behind both kinds, are set out above.